HTTP (HyperText Transfer Protocol)

1. Introduction to HTTP:

- HTTP stands for **HyperText Transfer Protocol**.
- It is an **application layer protocol** used for transmitting hypermedia documents such as HTML.
- It forms the **foundation of data communication** on the World Wide Web.
- Designed for **client-server communication** — clients (usually browsers) send requests, and servers respond.

2. Characteristics of HTTP:

- **Connectionless:** Each HTTP request is independent; a new connection is opened for every request.
- **Stateless:** The server does not retain information about previous client requests.
- **Media Independent:** Any type of data can be transferred, provided the client and server can handle it.
- **Extensible:** Allows new methods, headers, and features to be added over time.

3. HTTP Architecture:

- **Client:** Sends HTTP requests to the server (usually a web browser).
- **Server:** Processes the request and sends an HTTP response (web server software like Apache, Nginx).
- **Connection:** Usually over **TCP/IP** —
  - Port **80** for HTTP
  - Port **443** for HTTPS

4. HTTP Request Structure:

An HTTP request consists of:

1. **Request Line** – Contains:
    a. Method (GET, POST, etc.)
    b. Resource (URL)
    c. Protocol version (e.g., HTTP/1.1)
    Example:
    GET /index.html HTTP/1.1
2. **Headers** – Provide additional information about the request: Host, User-Agent, Accept, Content-Type, etc.
3. **Body** – Optional part that carries data to the server (used mainly in POST, PUT).

5. HTTP Response Structure:

An HTTP response consists of:

1. **Status Line** – Includes HTTP version, status code, and status message.
    Example: HTTP/1.1 200 OK
2. **Headers** – Provide metadata about the response: Content-Type, Content-Length, Server, etc.
3. **Body** – Contains the requested resource (HTML, JSON, image, etc.).

6. Common HTTP Methods

- **GET:** Retrieve data from the server.
- **POST:** Send data to the server for processing or storage.
- **PUT:** Update an existing resource.
- **DELETE:** Remove a resource.
- **HEAD:** Same as GET but returns only headers.
- **OPTIONS:** Describes communication options available.

7. HTTP Status Codes

Category	Code	Meaning	Example
1xx	Informational	Request received and continuing	100 Continue
2xx	Success	Request successful	200 OK, 201 Created
3xx	Redirection	Further action needed	301 Moved Permanently
4xx	Client Error	Request has a problem	404 Not Found, 403 Forbidden
5xx	Server Error	Server failed to fulfill request	500 Internal Server Error

8. Persistent vs Non-Persistent Connections

- **Non-Persistent:**
  - One TCP connection per request/response pair.
  - Increases overhead due to multiple connections.
- **Persistent (Keep-Alive):**
  - Same TCP connection used for multiple requests.
  - Reduces latency and improves performance.

9. HTTPS (Secure HTTP)

- HTTPS = HTTP + SSL/TLS (Secure Sockets Layer / Transport Layer Security).
- Ensures:
  - **Encryption:** Protects data from eavesdropping.
  - **Integrity:** Prevents tampering.
  - **Authentication:** Verifies server identity.
- Uses **port 443** instead of 80.

10. HTTP/1.1 vs HTTP/2 vs HTTP/3

Version	Characteristics
HTTP/1.1	Text-based, only one request per connection, introduced persistent connections.
HTTP/2	Binary protocol supports multiplexing (multiple requests in one connection), header compression.
HTTP/3	Uses QUIC protocol over UDP, improves speed, security, and reliability.

11. Cookies, Sessions, and Caching

- **Cookies:**
  - Small pieces of data stored on the client by the server.
  - Used for remembering user preferences, authentication, etc.
- **Sessions:** Stored on the server, maintaining user state across multiple requests.
- **Caching:** Temporary storage of web data to reduce server load and response time.

12. Real-World Applications of HTTP

- Web browsers use HTTP to fetch and display web pages.
- APIs (Application Programming Interfaces) use HTTP/HTTPS for communication (RESTful APIs).
- File transfer systems, IoT devices, and web applications all rely on HTTP methods for data exchange.

Summary

- HTTP is a **stateless protocol** used for transferring data between clients and servers.
- HTTPS adds **security** through encryption and authentication.
- Modern web systems use advanced protocols (**HTTP/2, HTTP/3**) to improve speed and reliability.
- Understanding HTTP is fundamental for **web development, cybersecurity, and networking**.